SIMATS ENGINEERING

ASSESSMENT TOOLS

COURSE NAME: Theory of Computation
COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL: 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks: 25**

Duration : 1 Hour

Design Critique Session

Code of Conduct:

I **J.Goutham Kumar Reddy/ 192311140 (**Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

J.Goutham Kumar Reddy

SIMATS ENGINEERING

ASSESSMENT TOOLS

1. A PDA is designed to recognize $a^n b^n$, but it fails for $a^n b^n c^n$. Critique the design and identify possible flaws in stack operations and state transitions. How would you improve it?

2. A student claims that Context-Free Grammars (CFGs)

E → E + T | T

T → T * F

| F F → (

E) | id

can be directly converted into a Deterministic PDA (DPDA). Critically evaluate this statement with justification and suggest corrections if needed.

3. **A Turing Machine is designed using multiple tracks to solve a problem. Suggest alternative programming techniques like checking-off symbols or subroutines to do subtraction.**

4. **While designing a PDA accepts the**

language L = {w | w ∈ (a + b)* and

n_a(w) == n_b(w)}

n_a(w) is the number of a's in w

n_b(w) is the number of b's in w, acceptance is defined only by final state, ignoring empty stack acceptance. Critique this design choice. Would it affect the language recognized? Explain with reasoning.

5. **A system designer suggests replacing a Turing Machine with a PDA to reduce computational complexity. Critically compare FA, PDA, and Turing Machine capabilities and evaluate whether this replacement is valid.**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (5)	Level 3 (4)	Level 2 (2-3)	Level 1 (0-1)
Conceptual Understanding	30	Clear & complete understanding	Good understanding, minor gaps	Basic understanding, some misconceptions	Poor / incorrect understanding
Accuracy of Answer	25	Completely correct, no errors	Mostly correct, minor errors	Partially correct, noticeable errors	Incorrect / irrelevant
Application of Knowledge	20	Correctly applies concepts	Applies with minor mistakes	Limited / partially correct application	No / incorrect application
Completeness of Response	15	Covers all required parts	Covers most parts, minor omissions	Covers only some parts	Incomplete / missing
Clarity & Presentation	10	Clear, concise, well-organized	Understandable, minor issues	Somewhat unclear / poorly structured	Difficult to understand

ANSWERS

1 — Critique: PDA for $a^n b^n$ fails on $a^n b^n c^n$

Why the original design fails

A PDA for $a^n b^n$ typically has the structure: push a stack symbol X for every a, then pop one X for every b, and accept when the stack returns to just Z0. The core flaw when this same machine is pointed at $a^n b^n c^n$ is:

- Stack operations only encode ONE counting relationship (a's vs. b's). A pushdown store is a single LIFO structure — it can reliably compare at most two quantities against each other (push for one symbol class, pop for the other).
- When c's start arriving, the stack is already empty (all X's popped by the b's), so there is no stack-based mechanism left to verify n_c equals n_a and n_b. The machine has no memory of how many a's/b's there were once the stack empties.
- State transitions compound the problem: a typical 2–3 state design (push-state → pop-state → accept-state) has no THIRD phase capable of matching c's against a remembered count, and no non-determinism can conjure that count back once it has been discarded from the stack.

Root cause (theoretical)

This is not a fixable implementation bug — it is a fundamental limitation of pushdown automata. It is a classical pumping-lemma result that $L = \{a^n b^n c^n \mid n \geq 1\}$ is NOT context-free, so NO pushdown automaton (however cleverly designed) can accept it. A single stack cannot simultaneously verify two independent equality constraints (a=b and b=c) because comparing the second pair requires the count that the first comparison already consumed.

How to improve / correct next steps

- Recognize the class boundary: $a^n b^n \in$ CFL (correctly solved by a 1-stack PDA); $a^n b^n c^n \notin$ CFL, so a PDA is the wrong model of computation for it altogether.
- Move to a more powerful model: a (single-tape, multi-pass) Turing Machine easily recognizes $a^n b^n c^n$ using the classical 'mark-and-sweep' technique — cross off one a, one b and one c per pass using extra tape symbols (X, Y, Z), repeating until all three groups are exhausted simultaneously.
- If restricted to pushdown-like devices, a 2-stack PDA (equivalent in power to a Turing Machine) COULD solve it — one stack matches a's against b's, and information can be relayed to a second stack to match against c's — but this is no longer a standard (1-stack) PDA.

In short: the flaw is not in a specific transition or stack symbol choice — it is that the problem has outgrown the computational power of the pushdown automaton model, and the fix is to select the correct model (Turing Machine) rather than to patch the PDA.

2 — Critique: 'CFG for arithmetic expressions converts directly to a DPDA'

Evaluating the claim

The grammar given is the classical unambiguous, left-recursive grammar for arithmetic expressions:

$E \rightarrow E + T \mid T,\quad T \rightarrow T * F \mid F,\quad F \rightarrow (E) \mid id$

The student's claim conflates two distinct facts and is only PARTIALLY correct:

- TRUE: The language generated by this grammar IS deterministic context-free (it is even LR(1)), so SOME deterministic PDA exists for it.
- FALSE / MISLEADING: The STANDARD textbook CFG-to-PDA construction (the one used in Answer 2 of the constraint-based paper, pushing the start symbol and expanding productions non-deterministically on the stack) always produces a NON-deterministic PDA — it works by GUESSING which production to apply at each step (e.g., guessing between E→E+T and E→T with no lookahead). That construction does NOT automatically yield a DPDA, regardless of how 'good' the grammar is.

Why the standard construction is non-deterministic here

- At variable E, the PDA must choose between E→E+T and E→T without having read any further input — this is a genuine non-deterministic branch (both are ε-productions on the stack, indistinguishable from the input read so far).
- Left recursion (E→E+T) is particularly troublesome: naively expanding it can even lead to non-termination in a top-down parser/PDA without careful handling.

Correction / how to actually obtain a DPDA

- The grammar must first be shown to be LR(1) (or at least deterministic context-free), then a DPDA can be built directly from the shift-reduce (LR) parsing table for the grammar rather than from the naive top-down construction. This grammar IS LR(1), so this correction succeeds.
- Alternatively, eliminate left recursion and left-factor the grammar, then build a predictive (LL) top-down DPDA using FIRST/FOLLOW sets to make each production choice deterministic with one symbol of lookahead.
- General caution for the student: not every CFG is deterministic context-free (e.g., ambiguous grammars, or languages requiring genuine backtracking such as palindromes ww^R) — for those, NO DPDA exists at all, even though a (non-deterministic) PDA does. The determinism must be established at the LANGUAGE level, and the DPDA constructed via LR-parsing techniques or left-factoring, not by assuming the generic CFG→PDA algorithm is deterministic.

Conclusion: the claim is correct only in the weak sense that a DPDA exists for this particular (LR(1)) grammar's language — it is incorrect as stated because the naive/'direct' CFG-to-PDA conversion procedure itself does not produce a deterministic machine; an LR-table-based or left-factored construction is required.

3 — Alternatives to Multi-Track Turing Machines

Multi-track TMs are a convenient DESIGN abstraction, but they are not more powerful than a standard single-track TM — every multi-track machine can be simulated by an ordinary one-track TM (each cell's k-tuple of track symbols is simply encoded as one symbol from a larger tape alphabet). The question asks for equally powerful, often more intuitive, single-track programming techniques.

Technique 1 — Checking-off (marking) symbols

Instead of a dedicated 'comparison track', reuse the single tape and mark symbols as they are processed by
overwriting them with a primed/marked version (e.g., a → X, b → Y). This is exactly the mark-and-sweep technique used for $a^n b^n c^n$ in Answer 1/5 of the constraint-based paper:

- Scan right, find the first un-checked symbol of type 1, mark it.
- Continue right, find the corresponding first un-checked symbol of type 2, mark it.
- Return (move left) to the start and repeat, until either a mismatch is found (reject) or all symbols are checked off together (accept).
- This achieves, on ONE track, exactly what a second dedicated 'marker track' would have done — at the cost of extra passes over the tape rather than extra tape alphabet dimensions.

Technique 2 — Subroutines (e.g., for subtraction)

Rather than hard-wiring a subtraction operation across multiple tracks (one track per operand), treat the two unary numbers as blocks of a symbol (e.g., 1's) separated by a marker, and implement subtraction as a reusable 'subroutine' — a self-contained block of states that can be entered from many places in the main machine and always returns control to the calling state, just like a function call:

- SUBROUTINE Decrement-Both: repeatedly cross off the left-most un-marked 1 in operand A and the left-most un-marked 1 in operand B on the SAME pass (checking off technique), moving the head back to the start of the whole block after each round.
- Termination conditions: if A runs out of un-marked 1's while B still has some → result 'A < B'; if B runs out while A still has some → the number of leftover 1's in A IS the subtraction result (A − B); if both run out together → result is 0.
- Because this block of states is entered/exited in a disciplined way (fixed entry state, fixed set of 'return' states depending on the outcome), it can be invoked from several places in a larger TM design (e.g., inside a multiplication or division TM that repeatedly subtracts), exactly like a subroutine call in ordinary programming — avoiding the need to duplicate the subtraction logic or dedicate a permanent extra track to it.

Why these are preferable to multi-track designs

- Fewer tape symbols overall (the alphabet doesn't grow combinatorially with the number of 'tracks' needed), which keeps the transition table smaller and easier to verify.
- Checking-off and subroutines are compositional — the same subtraction subroutine can be reused unmodified inside a larger machine (e.g., division-by-repeated-subtraction), whereas a multi-track design tends to be bespoke to one problem.
- All three techniques (multi-track, checking-off, subroutines) are provably equivalent in computational power — the choice is purely about design clarity and reusability,

not capability.

4 — Critique: Final-State-Only Acceptance for $n_a(w) = n\_b(w)$

Does the acceptance mode matter in theory?

No — this is one of the fundamental equivalence theorems of PDA theory: acceptance by final state and acceptance by empty stack define exactly the same class of languages (the context-free languages). For every PDA that accepts by empty stack there is an equivalent PDA that accepts by final state, and vice versa (via standard, mechanical conversions). So, IN PRINCIPLE, choosing final-state acceptance instead of empty-stack acceptance cannot, by itself, change which language is recognized.

Where the real risk lies — the IMPLEMENTATION, not the acceptance mode

The critique should focus on how the final-state design is actually built, because switching to final-state acceptance removes the 'free safety check' that empty-stack acceptance provides:

- With empty-stack acceptance, the machine is automatically forced to have used up every pushed symbol before the input is exhausted — an unbalanced string can never empty the stack, so the check is built in.
- With final-state acceptance, the designer must explicitly verify that the stack is properly balanced before transitioning into the final state. A careless design could enter the 'accepting' state while extra unmatched symbols (leftover a's or b's worth of stack content) are still sitting below the top of the stack — this would incorrectly ACCEPT unbalanced strings, or the designer might forget a transition and incorrectly REJECT some balanced strings.
- Concretely, for $n_a(w)=n\_b(w)$, a correct final-state design must still ensure the stack has returned to just Z0 (or an equivalent 'balanced' marker) at the moment the final state is entered — e.g., by adding an explicit ε-transition of the form $\delta(q, \varepsilon, Z0) = (q\_final, Z0)$ that only fires when the counting stack symbol is fully cancelled, mirroring the empty-stack check rather than skipping it.

Conclusion

The choice of final-state vs. empty-stack acceptance does NOT, by itself, change the language recognized — both modes are formally equivalent in expressive power for PDAs. The genuine design risk is practical, not theoretical: without the automatic discipline that empty-stack acceptance provides, the designer must manually re-encode the 'stack is balanced' condition into the transition to the final state, and any oversight there — not the choice of acceptance mode — is what would actually cause the machine to mis-recognize the language.

5 — Critique: Replacing a Turing Machine with a PDA

Capability comparison

Aspect	Finite Automaton (FA)	Pushdown Automaton (PDA)	Turing Machine (TM)
Memory	None (fixed states)	One stack (LIFO)	Unbounded tape, read/write, moves both ways
Language class	Regular languages	Context-free languages	Recursively enumerable (includes CFLs, regular)
Can count/compare	Nothing (bounded)	Two quantities (e.g. n_a=n_b)	Arbitrarily many quantities, arbitrary computation
Cannot handle	a^nb^n	$a^nb^nc^n$	(none — limited only by undecidability)
Real-world analogy	Lexical scanners, regex	Syntax parsers (bracket matching)	General-purpose computers/algorithms

Is 'PDA instead of TM' a valid substitution?

It depends entirely on the language/problem the system actually needs to solve — the hierarchy FA ⊂ PDA ⊂ TM is strict (each is a proper subset of the next in terms of expressive power), so a PDA can NEVER simulate everything a TM can:

- VALID replacement: if the task is fundamentally a context-free parsing problem (e.g., checking balanced parentheses, parsing a programming language's syntax, validating a^nb^n-style nesting) — a PDA is not just adequate, it is the theoretically appropriate and typically more efficient/lighter-weight choice; using a full TM there is overkill, not an error.
- INVALID replacement: if the task requires unbounded, arbitrary computation, or needs to compare/count MORE than two independent quantities (e.g., $a^nb^nc^n$, general arithmetic/algorithmic computation, anything requiring random access to previously written data) — a PDA is mathematically incapable of the task, regardless of implementation cleverness. The stack's LIFO discipline and the PDA's inability to re-read/modify arbitrary past symbols are hard, provable limitations, not engineering shortcomings.

On the 'reduce computational complexity' justification

A PDA is not simply a 'faster/lighter Turing Machine' — it is a strictly less expressive model. If the underlying language IS context-free, using a PDA can indeed be more efficient (e.g., O(n) or $O(n^3)$ parsing vs. simulating a general TM), so complexity gains are real and legitimate in that case. But if the language is NOT context-free, there is no PDA that decides it at any complexity — the substitution would silently make certain valid or invalid inputs impossible to classify correctly, which is a correctness failure, not a performance trade-off.

Recommendation

Before approving the replacement, first classify the language of the actual problem: (1) confirm it is context-free (and ideally deterministic context-free, for an efficient DPDA/LR-parser implementation); (2) if so, the PDA replacement is valid and likely beneficial; (3) if the language requires counting/matching more than two dependent quantities, unbounded random-access memory, or general algorithmic computation, the TM (or an equivalent general-purpose

computational model) must be retained, and the proposed replacement should be rejected.